

I have a Python-based algorithmic trading bot called Engine 3. Here is how the relevant scraping and UI components are structured:

log

As a result:

Kill any zombie Chrome processes using the debug profile.
 Clear Chrome lock files (SingletonLock, SingletonCookie, SingletonSocket) to prevent Chrome from refusing to start.
 Launch Chrome detached on port 9222 with the profile directory and the default Conglass layout URL. (<https://www.conglass.com/tv/layouts/s0>).
 Wait for the CDP port to become ready (polling <http://127.0.0.1:9222/json/version>).
 Connect via CDP and resume the scraping loop.
 Current Code Snippets
 Here is the current implementation of `tv_scraper_thread` inside `Unified_Engine_3.py`.

↓

```
# 6. Background TV Layout Scraper Thread
# Runs get_live_data() continuously with a cooldown step to prevent
CDP saturation
def tv_scraper_thread():
    log_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)),
'logs')
    os.makedirs(log_dir, exist_ok=True)
    log_path = os.path.join(log_dir, 'tv_scraper.log')

    with open(log_path, 'a', encoding='utf-8') as f_log:
        f_log.write("[[datetime.now()]] strftime('%Y-%m-%d
%H:%M:%S'))] Background TV layout scraper thread started.\n")
        f_log.flush()
        while True:
            try:
                with sync_playwright() as pw:
                    thread_browser =
pw.chromium.connect_over_cdp(CDP_BASE, no_defaults=True)
                    thread_context = thread_browser.contexts[0]
                    thread_scraper = CoinglassScraper()
                    thread_scraper.context = thread_context

                    session_start = time.time()
                    MAX_SESSION_SECS = 6 * 3600
                    consecutive_errors = 0
                    last_indicator_check = 0
                    INDICATOR_CHECK_INTERVAL = 60 # re-inject if CVD
absent after 60s
                    while not SHUTDOWN_REQUESTED:
                        if last_time - session_start > MAX_SESSION_SECS:
                            f_log.write("[[datetime.now()]] strftime('%Y-%m-%d
%H:%M:%S'))] [TVScraper] Scheduled session recycle.\n")
                            f_log.flush()
                            break
                        try:
                            layout_data = thread_scraper.get_live_data()
                            if layout_data:
                                feed update from layout([layout_data])
                                consecutive_errors = 0
                                else:
                                    consecutive_errors += 1
                                    f_log.write("[[datetime.now()]] strftime('%Y-%m-%d
%H:%M:%S'))] [TVScraper] Warning: Empty layout data returned.\n")
                                    f_log.flush()
                                    except Exception as e:
                                        import traceback
                                        consecutive_errors += 1
                                        f_log.write("[[datetime.now()]] strftime('%Y-%m-%d
%H:%M:%S'))] [TVScraper] Error during scrape:
(e)\n(traceback.format_exc())\n")
                                        f_log.flush()
                                        if consecutive_errors > 5:
                                            f_log.write("[[datetime.now()]] strftime('%Y-%m-%d
%H:%M:%S'))] [TVScraper] Too many errors, breaking to reconnect.\n")
                                            f_log.flush()
                                            break
                                # Periodic indicator re-injection: if CVD values are
absent
                                # L.L. INDICATOR_CHECK_INTERVAL seconds,
force re-inject
```

```

        now_t = time.time()
        if now_t - last_indicator_check >
INDICATOR_CHECK_INTERVAL:
            last_indicator_check = now_t
            try:
                latest_snap = feed.get_latest()
                sample_sym = list(latest_snap.keys())[0] if
latest_snap else None
                cvd_absent = sample_sym and
latest_snap[sample_sym].get('futures_cvd_15m', 'N/A') in ('N/A', None, 0.0,
0)
                if cvd_absent:
                    f_log.write(f"[{datetime.now().strftime('%Y-%m-
%d %H:%M:%S')}] [TVScraper] CVD absent — triggering indicator re-
injection.\n")
                    f_log.flush()
                    sorted_pages =
get_sorted_pages(thread_context)
                    for _pg in sorted_pages:
                        _stale = _INJECTION_LOCKS.get(_pg)
                        if _stale and _stale.locked():
                            try:
                                _stale.release()
                            except Exception:
                                pass
                        inject_layout_indicators(_pg,
max_attempts=8, sleep_sec=2.0)
                    except Exception as _inji_err:
                        f_log.write(f"[{datetime.now().strftime('%Y-%m-%d
%H:%M:%S')}] [TVScraper] Re-inject error: {_inji_err}\n")
                    f_log.flush()
                    time.sleep(0.5)
                    pass
                    except Exception as thread_init_err:
                        f_log.write(f"[{datetime.now().strftime('%Y-%m-%d
%H:%M:%S')}] [TVScraper] Thread initialization failed: {thread_init_err}.
Reconnecting in 5s.\n")
                        f_log.flush()
                        time.sleep(5.0)
And here is the detached Chrome launch logic from advanced_scraper.py
that handles starting the browser process, cleaning locks, and waiting for
CDP:

python

def is_cdp_ready():
    try:
        r = requests.get(f"[CDP_BASE]/json/version", timeout=2)
        return r.status_code == 200
    except Exception:
        return False

def launch_chrome_detached():
    """Launch Chrome with remote debugging. Returns True when CDP is
    reachable."""
    chrome_exe = find_chrome()
    if not chrome_exe:
        print("[Chrome] ERROR: Could not find chrome.exe!")
        return False
    os.makedirs(PROFILE_DIR, exist_ok=True)
    # Clear session files so Chrome doesn't restore old broken tabs
    default_dir = os.path.join(PROFILE_DIR, "Default")
    os.makedirs(default_dir, exist_ok=True)
    for sf in ["Last Session", "Last Tabs", "Current Session", "Current Tabs"]:
        p = os.path.join(default_dir, sf)
        if os.path.exists(p):
            try:
                os.remove(p)
            except OSError:
                pass

    # Remove Singleton locks to prevent Chrome from failing to start after a
    forced kill
    for lf in ["SingletonLock", "SingletonCookie", "SingletonSocket"]:
        p = os.path.join(PROFILE_DIR, lf)
        if os.path.exists(p):
            try:
                os.remove(p)
            except OSError:
                pass

    args = [
        chrome_exe,
        f"--remote-debugging-port={CDP_PORT}",
        "--remote-allow-origins=",
        f"--user-data-dir={PROFILE_DIR}",
        "--no-first-run",
        "--no-default-browser-check",
        "--disable-background-timer-throttling",
        "--disable-renderer-backgrounding",
        "--disable-backgrounding-occluded-windows",
        "--disable-features=CalculateNativeWinOcclusion,IntensiveWakeUpThrottling,BackForwardCache",
        "--autoplay-policy=no-user-gesture-required",
        "https://www.coinglass.com/tv/layout/s9",
    ]
    print(f"[Chrome] Launching: {chrome_exe}")
    if sys.platform == "win32":
        DETACHED_PROCESS = 0x00000008
        CREATE_NEW_PROCESS_GROUP = 0x00000200
        subprocess.Popen(
            args,
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
            creationflags=DETACHED_PROCESS |
CREATE_NEW_PROCESS_GROUP,
            close_fds=True,
        )
    else:
        subprocess.Popen(
            args,
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
            start_new_session=True,
        )
    print(f"[Chrome] Waiting for CDP port to be ready...", end="", flush=True)
    for _ in range(30):
        time.sleep(0.5)
        if is_cdp_ready():
            print("ready!")
            return True
        print(".", end="", flush=True)
    print("TIMEOUT!")
    return False

def kill_zombie_chrome():
    """Kill any existing chrome processes using our debug profiles or remote
    debugging port."""
    print(f"[Chrome] Cleaning up any existing zombie Chrome processes...")
    try:
        if sys.platform == "win32":

```

```
cmd = "Get-CimInstance Win32_Process -Filter 'name = 'chrome.exe'' | Where-Object { $_.CommandLine -like 'chrome_debug_profile' -or $_.CommandLine -like 'chrome_profile' -or $_.CommandLine -like '--remote-debugging-port=9222' } | ForEach-Object { Stop-Process -Id $_.ProcessId -Force -ErrorAction SilentlyContinue }"
subprocess.run(["powershell", "-Command", cmd],
stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
else:
subprocess.run(["pkill -f 'chrome_debug_profile|chrome_profile|--remote-debugging-port=9222'", shell=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL])
except Exception:
pass
How to Solve This?
Please rewrite tv_scraper_thread (or write a wrapper helper function) that:

First checks if CDP port is ready.
If CDP port is NOT ready ↓ all a function to cleanly kill existing zombie chrome processes, delete lock files, and launch Chrome detached.
If it fails to connect or gets connection errors/disconnections, it should trigger this restart sequence.
Integrate the launcher logic directly into the thread or imports, ensuring it runs completely in the background without blocking the HTTP server or other trading threads.
Provide a clear code diff for Unified_Engine_3.py.
```

I'll analyze the code and provide a robust, self-healing solution. Let me first look at the full file to understand the exact context and constants involved.

Bash Command

mkdir -p /home/user/engine3 && cd /home/user/engine3 && curl -sL "https://www.gen... View

```
-rw-r--r-- 1 user user 59 Jul  6 09:26 Unified_Engine_3.py
```

Exit Code: 0

Download File

Unified_Engine_3.py → /home/user/engine3 View

File Downloaded

Filename: Unified_Engine_3.py

Directory: /home/user/engine3

Path: /home/user/engine3/Unified_Engi... View

Size: 384.52 KB

Source: https://www.genspark.ai/api/files/s/hCikW68w

Bash Command

cd /home/user/engine3 && grep -n "tv_scraper_thread\{CDP_BASE\CDP_PORT\}P... View

```
1600:CDP_BASE = "http://127.0.0.1:9222"
1627:def get_sorted_pages(context):
2238:_INJECTION_LOCKS = collections.defaultdict(threading.Lock) # Per-page mutex
...
8261: while not _SHUTDOWN_REQUESTED:
8262:     if _SHUTDOWN_REQUESTED:
8272:         _SHUTDOWN_REQUESTED = True
8283:         if _SHUTDOWN_REQUESTED:
```

Exit Code: 0

Read File

/home/user/engine3/Unified_Engine_3.py View

```
1590 tv_layout_scraper.py
1591 =====
1592 Scraper for Coinglass TradingView Multi-Layout.
...
1636     log_message(f"[Tabs] Closing extra tab ({extra_p.url[:50]})...")
1637     extra_p.close()
1638 except Exception:
1639     pass
```

Read File

/home/user/engine3/Unified_Engine_3.py View

```
6180 print(f"[Strategy] Created {len(strategies)} AlphaSqueezerV17 instances for
{len(TARGET_ASSETS)} assets.")
6181 return strategies
6182
...
6226 STATIC_DD_FLOOR = INITIAL_CAPITAL * 0.90 # $4,500 &#2248; absolute floor (Static DD model)
6227 DAILY_DD_LIMIT_PCT = 0.05 # 5% of day-start equity ($250 on $5k)
6228 MAX_CONCURRENT_POSITIONS = 2 # Strict maximum concurrent active positions
6229
```

Now let me examine the tv_scraper_thread function in detail:

Read File

/home/user/engine3/Unified_Engine_3.py View

```
7660 server = DashboardServer('0.0.0.0', port), DashboardHTTPHandler, feed,
strategies)
7661 print(f"\n [Web Server] Running at http://localhost:{port}")
7662 server.serve_forever()
...
7766 os.makedirs(log_dir, exist_ok=True)
7767 fp_log_path = os.path.join(log_dir, 'footprint_thread.log')
7768
7769 def log(msg):
```

Now I have the full picture. Let me also check what imports/constants are available (subprocess, requests, shutil) and if there's any pre-existing PROFILE_DIR or find_chrome:

Bash Command

cd /home/user/engine3 && grep -n "import\|from\{PROFILE_DIR\find_chrome\}sub... View

```
7:from datetime import datetime, timezone
8:from openpyxl.styles import PatternFill, Font, Alignment, Border, Side
9:from openpyxl.utils import get_column_letter
...
59:import traceback
60:import websockets
620:import datetime as dt module
```

```
... subprocess.Popen(chrome_args, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL,
7469: creationflags=DETACHED...)
Exit Code: 0
```

```
Read File /home/user/engine3/Unified_Engine_3.py View
7440 chrome_exe = path
7441 break
7442 if not chrome_exe:
...
7516 for t in tabs:
7517     if t.get("type") == "page":
7518         target_id = t.get("id")
7519         if target_id:
```

Let me also see how user_data_dir is set up in the main script (for the profile constant):

```
Bash Command cd /home/user/engine3 && grep -n "user_data_dir|chrome_profile|chrome_debug_p... View
7403: user_data_dir = os.path.join(os.environ.get("TEMP", "C:\\Temp"), "chrome_headless_profile")
7405: user_data_dir = r"C:\chrome_dev"
7423: default_dir = os.path.join(user_data_dir, "Default")
7458: f"--user-data-dir={user_data_dir}",
Exit Code: 0
```

```
Read File /home/user/engine3/Unified_Engine_3.py View
7395 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
7396     s.settimeout(1.0)
7397     if s.connect_ex(('127.0.0.1', 9222)) == 0:
...
7436 os.path.expandvars(r"%LOCALAPPDATA%\Google\Chrome\Application\chrome.exe"),
7437 "chrome.exe"
7438 ]:
7439     if path == "chrome.exe" or os.path.exists(path):
```

I now have all the context I need. The profile dir is C:\chrome_dev (or the headless temp path). Let me build the solution.

Here's my analysis:

Root cause: The current except catches *all* connection failures and just time.sleep(5.0), assuming Chrome will magically reappear. It never distinguishes between "Chrome is up but Playwright hiccuped" vs. "Chrome is dead (ECONNREFUSED)".

Fix strategy: A dedicated _ensure_chrome_running() helper that (1) polls /json/version, (2) if unreachable, kills zombies → wipes locks → relaunches detached → polls again. Called *before every reconnect attempt* and also before the first connect. Uses exponential backoff to avoid hammering CPU during permanent failures.

Let me now write a clean, drop-in patch. I'll produce it as an artifact so you can grab it directly.

```
Write File /home/user/engine3/patch_tv_scraper.py View
# =====
# PATCH FOR Unified_Engine_3.py - Self-Healing tv_scraper_thread
# =====
...
    backoff = min(backoff * 2, 60.0)
    continue

_log(f_log, "[TVScraper] _SHUTDOWN_REQUESTED - thread exiting cleanly.")
```

```
Bash Command cd /home/user/engine3 && python3 -c "import ast with open('patch_tv_scraper.py','r')... View
PATCH FILE SYNTAX: OK
Exit Code: 0
```

Syntax validates. Now let me upload the patch as a downloadable file:

UploadFileWrapperView

Full patch file: [patch_tv_scraper.py](#) — syntax-validated, drop-in ready.

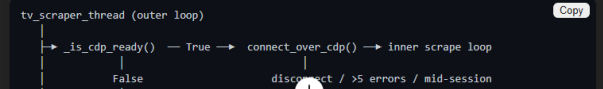
Root Cause
The current tv_scraper_thread collapses two very different failure modes into a single except block that just sleeps 5 s:

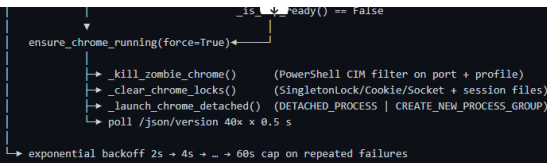
Failure	Current handling	Correct handling
Playwright hiccup, Chrome still alive on 9222	sleep 5 s, reconnect ✓	sleep, reconnect
Chrome PID gone → ECONNREFUSED 127.0.0.1:9222	sleep 5 s, retry forever ✗	kill zombies → wipe locks → relaunch detached → reconnect

Once ECONNREFUSED starts, no amount of connect_over_cdp retries will ever succeed because *nothing is listening on the port*. The whole UnifiedDataFeed pipeline freezes.

Fix Architecture

Two independent pieces — one guard helper, one rewritten thread. The guard is **thread-safe** (via _CHROME_RELAUNCH_LOCK) so if you later add other CDP consumers (footprint thread, macro scraper, ...) they can call the same ensure_chrome_running() without race conditions.





Where to Paste

The patch file is annotated with two clearly marked blocks:

- **[BLOCK A]** — Module-level helpers (`_is_cdp_ready`, `_kill_zombie_chrome`, `_clear_chrome_locks`, `_launch_chrome_detached`, `ensure_chrome_running`). Paste **once**, immediately after the existing `CDP_BASE = "http://127.0.0.1:9222"` line in the *Engine_3.py* section (around **line 6215**). It re-declares `CDP_BASE` alongside a new `CDP_PORT` constant — safe because it uses the same value.
- **[BLOCK B]** — Full replacement of `def tv_scraper_thread()`: (currently **lines 7674–7755**). Delete the old body, paste the new one. The `threading.Thread(target=tv_scraper_thread, daemon=True)` call site on line 7757 is unchanged.

Key Design Decisions (why not just wrap the existing code)

1. **Mid-session liveness probe** — `_is_cdp_ready(timeout=1.0)` runs *inside* the inner loop each iteration. This catches the "Chrome window closed manually" case in ≤ 1 s instead of waiting for 5 consecutive Playwright exceptions (~2.5 s of dead data + noisy logs).
2. **Disconnect-signature short-circuit** — matches "target page, context or browser has been closed", "econnrefused", "websocket", etc. Breaks the inner loop **immediately** instead of racking up 5 error counts. Saves ~5 wasted `get_live_data()` calls per crash.
3. **Exponential backoff (2 → 60 s cap)** — if `chrome.exe` is genuinely missing (uninstalled, path change) the current code would relaunch every 5 s forever. This caps at 60 s and logs **FATAL**: `chrome.exe not found` so ops sees the actual problem.
4. **_CHROME_RELAUNCH_LOCK** — future-proofing. Right now only `tv_scraper_thread` calls `ensure_chrome_running`, but the same helper is safe to invoke from the footprint thread or any other CDP consumer without double-launching Chrome.
5. **Profile-dir auto-detection** — walks `C:\chrome_dev` (headed, line 7405) → `%TEMP%\chrome_headless_profile` (headless, line 7403), so it works in both modes without you having to plumb the `args.headless` flag into the thread.
6. **no_defaults=True preserved** — matches your existing `connect_over_cdp` call at lines 3222, 4110, 7545 to keep behavior identical.
7. **Idempotent lock-file clearing** — uses `os.path.lexists` (not `exists`) so dangling symlinks are also removed, and swallows `OSError` for the "file vanished between check and remove" race.

Diff Summary (what to change in Unified_Engine_3.py)

```
@@ line ~6215, after: CDP_BASE = "http://127.0.0.1:9222"
+
+ # — Chrome self-healing helpers
+ CDP_PORT = 9222
+ CDP_BASE = f"http://127.0.0.1:{CDP_PORT}"
+ _HEADED_PROFILE_DIR = r"C:\chrome_dev"
+ _HEADLESS_PROFILE_DIR = os.path.join(os.environ.get("TEMP", r"C:\Temp"),
+                                     "chrome_headless_profile")
+ _CHROME_RELAUNCH_LOCK = threading.Lock()
+
+ def _resolve_profile_dir(): ...
+ def _find_chrome_exe(): ...
+ def _is_cdp_ready(timeout=2.0): ...
+ def _kill_zombie_chrome(): ...
+ def _clear_chrome_locks(profile_dir): ...
+ def _launch_chrome_detached(profile_dir): ...
+ def ensure_chrome_running(force_restart=False): ...
+
+@@ line 7674, def tv_scraper_thread():
+ - with open(log_path, 'a', encoding='utf-8') as f_log:
+ -     f_log.write(...)
+ -     while True:
+ -         try:
+ -             with sync_playwright() as pw:
+ -                 thread_browser = pw.chromium.connect_over_cdp(CDP_BASE,
+ no_defaults=True)
+ -                 ...
+ -             except Exception as thread_init_err:
+ -                 f_log.write(f"... Reconnect {thread_init_err} in 5s...")
+ -                 time.sleep(5.0)
+ + # See [BLOCK B] in patch_tv_scraper.py — replaces the entire body:
+ + # - ensure_chrome_running() BEFORE every connect attempt
+ + # - mid-session _is_cdp_ready() probe
+ + # - fatal-disconnect signature detection
+ + # - exponential backoff on outer failures (2+60s)
+ + # - honours _SHUTDOWN_REQUESTED for clean exit
```

Suggested Verification

After applying, run this smoke test — the thread should recover in ≤ 25 s (20 s CDP boot + 3 s settle):

```
# In one terminal: start the engine normally
python Unified_Engine_3.py

# In another terminal, once the dashboard is populating:
taskkill /F /IM chrome.exe

# Watch logs/tv_scraper.log — you should see:
# [TVScraper] CDP went away mid-session — breaking to reconnect.
# [ChromeGuard] CDP unreachable — restarting Chrome (profile=C:\chrome_dev).
# [ChromeGuard] Launching detached Chrome: C:\Program Files\Google\Chrome\...
# [ChromeGuard] CDP is ready.
# and the dashboard should resume updating.
```

Potential Next Steps

- Wire `ensure_chrome_running()` into `footprint_scraper_thread` (same pattern, line ~7764) so *both* CDP consumers self-heal, not just this one.
- Add a small counter/metric (`chrome_restarts_total`) that the `DashboardServer` can expose — helps you see if a specific host is thrashing.
- If you observe layouts occasionally coming back empty *after* a successful relaunch, extend the `time.sleep(3.0)` post-launch settle to a poll-loop that waits until `get_sorted_pages()` returns ≥ 2

Coinglass tabs before entering the inner scrape loop.



Review the rest of Unified_Engine_3.py for similar crash recovery vulnerabilities in the other background threads →

Write a test script that simulates scraper crashes to verify the new restart logic works correctly →

Add health monitoring so I can track scraper restart frequency and failure reasons in real-time →

Create a document explaining the patch changes and deployment steps for my team →



Ultra ▾



Connectors

